Project：基于 MCP 的操作系统状态监控与分析助手

1. 项目概述

构建一个通过自然语言交互的智能操作系统诊断平台。通过开发相关 MCP Server，让大模型能够通过自然语言查询来监控操作系统的核心状态，包括内存使用、文件系统、进程管理等，将 openEuler 操作系统的底层状态和核心原理以直观、可交互的方式呈现出来。

2. 项目目标

（1） 深化操作系统原理理解：通过亲手实现与操作系统核心组件（内存、文件、进程）的交互，将抽象的理论知识（如页面置换、磁盘分配）转化为具体的、可观察的系统行为。

（2） 掌握现代 AI 工程范式：学习并实践模型上下文协议（Model Context Protocol，MCP），理解如何让大模型结构化地调用系统工具，构建领域专用的 AI Agent。

3. 项目内容

3.1 任务一：MCP 框架搭建

（1） 学习目标

a) 理解 MCP（Model Context Protocol）协议的工作原理
b) 掌握创建、配置和运行一个基础 MCP Server 的方法
c) 建立项目的基础工程结构

（2） 任务内容

a) 创建标准的项目结构。编写 requirements.txt，包含 mcp，pydantic，psutil 等核心依赖；编写项目配置文件。
b) 基础 MCP Server 框架。实现一个基础的 MCP Server, 实现 echo 或 get_time 工具，用于验证通信链路。
c) 客户端链接测试。将该基础 Server 成功连接到 EulerCopilot 或 Claude Desktop，确保可以通过自然语言调用基础工具并获得正确响应。

3.2 任务二：操作系统核心 MCP Server 开发

3.2.1 模块一：内存管理 MCP Server

（1） 学习目标

a) 掌握虚拟内存管理机制，包括分页、页面置换算法
b) 分析进程地址空间布局和内存映射关系

c) 理解内存访问模式与局部性原理

（2） 功能设计——MCP 协议接口实现

get_memory_info(): 返回系统内存使用情况 【操作系统原理】
get_process_memory(pid): 查询特定进程的内存映射【操作系统原理】
analyze_memory_patterns(): 分析内存访问模式【操作系统专题实验】
simulate_page_fault(scenario): 模拟页面置换算法【操作系统专题实验】

功能	实现思路	核心数据结构/命令
get_memory_info()	解析 /proc/meminfo, 使用 psutil.virtual_memory()	提取 MemTotal, MemFree, SwapCached 等
get_process_memory(pid)	解析 /proc/<pid>/smaps 或 pmap 命令	映射区域、权限、大小、RSS（驻留集大小）
analyze_memory_patterns()	周期性采样 /proc/pagetypeinfo 或模拟生成访问序列	计算缺页率、识别热点页面、分析时空局部性
simulate_page_fault(scenario)	实现 LRU、FIFO、CLOCK、OPT 等多种算法	维护页面帧栈/队列，模拟缺页中断处理

（3） 查询实例举例

[理解应用] 查询某一个特定进程在指定时间内内存使用变化情况，并返回映射方式。

[算法对比] “模拟一个工作负载，比较 FIFO 和 OPT 算法在 4 个物理帧下的缺页率。 ”

[程序局部性原理理解] “分析当前系统的内存访问模式，是否存在时间局部性或空间局部性？”

[理解内存布局] “给我看看 Nginx 进程的内存布局，指出代码段、数据段、堆、栈在哪里。”

[内存抖动理解]：启动多个内存密集型进程，周期性调用 get_process_memory(pid) 跟踪各自的驻留集大小（RSS）和映射区域变化。分析页面抖动（频繁换入换出）现象。

[页面置换理解]：在 Randmo，FIFO，LRU 之外，再设计一种可实现的置换算法，或者根据当前进程数量等情况，设计一个混合页面置换算法，比较这些算法的缺页率；并结合 simulate_page_fault(scenario)，输出详细换页日志和可视化结果。

3.2.2 模块二：文件系统 MCP Server

（1） 学习目标

a) 理解文件系统的磁盘分配策略（连续、链接、索引）及其权衡
b) 掌握文件存储分布分析和空间管理方法
c) 学会分析内存访问模式与局部性原理

（2） 功能设计——MCP 协议接口实现

get_filesystem_info(): 获取文件系统状态【操作系统原理】
analyze_file_distribution(): 分析文件分布模式 【操作系统原理】
trace_file_access(path): 跟踪文件访问路径【操作系统原理】
simulate_disk_allocation(): 模拟磁盘块分配策略【操作系统专题实验】

功能	实现思路	核心数据结构/命令
get_filesystem_info()	解析 df -h 和 df -i 输出	挂载点、总容量、使用率、inode 使用情况
analyze_file_distribution(path)	使用 find 和 du 命令，按扩展名、大小、时间归类	文件类型分布、大小分布、最近访问/修改时间
trace_file_access(path)	使用 inotify 机制或 auditd 系统调用	监控 open, read, write, close 事件
simulate_disk_allocation(scenario)	模拟三种分配策略的块分配过程	管理空闲块列表、文件分配表、索引块

（3） 查询实例举例
【算法对比】："模拟创建一系列文件，对比连续分配和索引分配产生的碎片情况；显示磁头移动轨迹与寻道开销。"
【空间分析】"根目录快满了，是哪个目录或文件类型占用了最多空间？"
【混合大小文件碎片演化】：使用 simulate_disk_allocation()，模拟在大量小文件+少量大文件并存场景下，不同分配策略（连续、索引等）的空间利用率、碎片增长速度；每隔一段时间用 get_filesystem_info() 和 analyze_file_distribution() 实时记录碎片化率、磁盘利用率变化曲线，并分析"写满-删除-重写"的循环对实际文件系统碎片的影响。

3.2.3 模块三：进程调度 MCP Server

（1） 学习目标
a) 理解深入理解进程状态、上下文切换和调度算法
b) 理解进程间关系（如父子关系）
c) 分析 CPU 利用率与调度延迟

（2） 功能设计——MCP 协议接口实现
get_process_tree(): 获取进程树关系【操作系统原理】
monitor_context_switches(): 监控上下文切换频率【操作系统原理】
simulate_scheduler(algorithm): 模拟不同调度算法【操作系统专题实验】
analyze_scheduling(): 分析调度策略效果【操作系统专题实验】

功能	实现思路	核心数据结构/命令
get_process_tree()	解析 pstree -p 或遍历 /proc	构建父子进程关系的树形结构

analyze_scheduling()	解析 /proc/stat 并计算差值	用户态、内核态、空闲、I/O 等待时间占比
monitor_context_switches()	解析 /proc/<pid>/sched 或使用 perf sched	进程的运行时间片、睡眠时间、自愿/非自愿上下文切换次数
simulate_scheduler(algorithm)	模拟实现 FCFS、SJF、Round-Robin 等算法	维护就绪队列，模拟时钟中断和调度决策

（3） 查询实例举例
[系统快照]: “展示当前的进程树，并高亮显示所有 Java 进程（可换）。”
[调度算法分析]: “模拟一个包含 CPU 密集型和 I/O 密集型任务的混合负载，使用时间片轮转调度，分析平均等待时间”
[资源分析]: “CPU 使用率为什么这么高？是用户态还是内核态开销”
[饥饿现象模拟]：扩展 simulate_scheduler()，实现带优先级提升/降低机制的调度器（如多级反馈队列等，参考恐龙书）；构造一组进程（高优先级短任务+低优先级长任务），观察低优先级进程是否出现饥饿现象；记录每个进程的等待时间、响应时间，讨论如何通过优先级调整缓解饥饿并初步验证加分项）。
[context switch 分析]：利用 monitor_context_switches()，在高并发（大量短任务）与低并发（少量长任务）场景下，分别监测单位时间上下文切换次数；对比两种场景下的 CPU 利用率、平均任务完成时间、系统负载；总结上下文切换的直接（CPU 浪费）与间接（缓存失效、延迟）影响，并给出优化建议，并初步验证（加分项）。

3.3 任务三：自然语言交互前端与系统集成

（1） 学习目标
a) 掌握如何管理和集成多个 MCP Server
b) 设计 AI agent 提示词和工作流
（2） 任务内容
a) 集成到 EulerCopilot 或自定义 AI 前端
b) 设计自然语言查询模版，实现智能诊断输出

4. 技术栈

- **操作系统**：openEuler 22.03+
- **编程语言**：Python 3.8+（主要），C（可选，用于内核模块）
- **核心库**：
  - mcp：MCP 协议实现
  - psutil：系统信息采集
- **可选择的开发工具**：
  - DevStation：在线开发环境
  - EulerCopilot：AI 助手集成

5. 交付物

（1） 可执行代码（项目打包）
项目应作为一个完整的、可部署的软件包进行提交。

（2） 实验报告（参见模版）
体现利用所开发的项目进行 openEuler 操作系统观察与诊断的实验过程和结果。